

Cloud Posture Dashboard – Technical Design

1.1 Problem Statement: Combating Cloud Security Alert Fatigue

Security teams are overwhelmed by the sheer volume of misconfiguration alerts generated across multi-cloud environments (AWS, Azure, GCP). This fragmentation leads to "alert fatigue," where thousands of warnings obscure critical risks. Lacking a unified and centralized perspective significantly delays threat detection and response times, causing critical issues to be overlooked by low-priority noise.

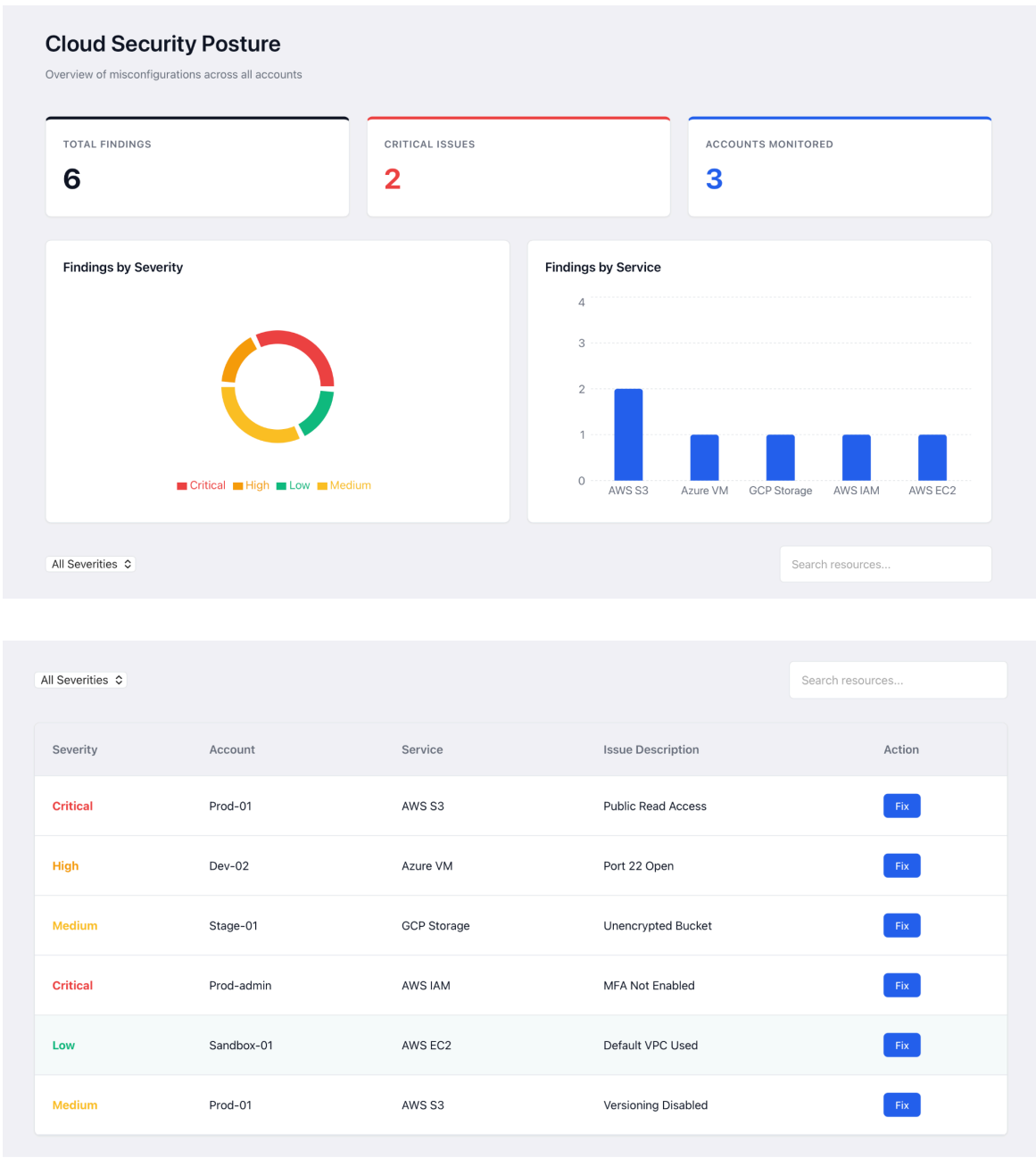
1.2 User Persona: Ahana: The Cloud Security Engineer

The application is specifically tailored for **Ahana**, a busy Cloud Security Engineer who requires quick, technical, and precise information.

Key Requirements:

- **Priority Filtering:** Ahana must be able to immediately filter data by **Critical Severity** and **Service**.
- **Efficiency:** She lacks the time to navigate complex UIs, multiple cloud consoles, or extensive data tables.
- **Technical Preference:** She is proficient with command-line tools (CLI) and favors direct, technical data over simplified interfaces ("wizards").
- **Core Goal:** To significantly cut down the "Time to Triage" from several minutes to a matter of seconds.

2.1 Figure 1: Unified Cloud Posture Dashboard



Description: This view addresses the user's need for high-level visibility. It features:

- **Top-Level Metrics:** Immediate insight into the total number of findings and critical issues.
- **Dynamic Visualization:** A severity distribution chart (Pie) and service breakdown (Bar) to help "Ahana" (the persona) orient himself to the scale of the risk.
- **Real-time Filtering:** Dropdown controls to instantly slice data by Severity or Service without reloading the page.

2.2 Figure 2: Remediation Workflow ("Human-in-the-Loop")

Remediate Issue

Issue: Public Read Access

Service: AWS S3 (us-east-1)

```
aws s3api put-public-access-block --bucket prod-01-bucket
--public-access-block-configuration
"BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy
```

This action will execute the CLI command above to resolve the misconfiguration.

Cancel

Confirm Fix

Description: This modal appears when the user clicks "Fix".

- **Safety First:** Instead of opaque automation, it displays the specific CLI command (e.g., aws s3api...).
- **Verification:** This allows the engineer to verify the action before execution, ensuring safety while maintaining speed.

3. Proposed Features & Prioritization

The proposed features for the dashboard were prioritized using the **OODA Loop (Observe, Orient, Decide, Act)** framework. This structured approach ensures the tool actively drives security outcomes rather than merely displaying data.

OODA Stage	Feature	Problem Solved	Solution Summary	Rationale	Priority
Observe	Global Severity Filtering	High volume of low-severity logs (noise) obscures critical security signals.	A client-side engine for instant filtering, focusing users on "Critical" and "High" issues.	The ability to find the 'signal' amidst the 'noise' is fundamental; without it, the dashboard is immediately ineffective.	P0
Orient	Visual Data Aggregation	A raw list of findings is difficult to interpret, making it hard to distinguish isolated vs. systemic issues.	Integrated Recharts to visualize Severity Distribution (Pie Chart) and Service Breakdown (Bar Chart).	Visuals enable rapid assessment of the attack surface (e.g., identifying that 80% of issues are concentrated in a single service like AWS S3).	P1
Act	Remediation Modal	Identifying a misconfiguration is unproductive without a direct workflow to fix it.	"Human-in-the-Loop" workflow that presents the specific CLI command (e.g., <code>aws s3api...</code>) for verification before execution.	This step closes the security loop, but was logically placed after visibility (P0) and context (P1), it must be seen and understood before fixing.	P2

4. Technical Implementation & Success Metrics

Technology Stack:

Component	Technology	Rationale
Frontend	React.js (Vite)	Chosen for superior component modularity and high-speed rendering.
Data Visualization	Recharts	Selected for its lightweight nature and composable API, enabling responsive charts without significant overhead.
State Management	React Hooks (useState, useMemo)	Utilized for efficient handling of client-side filtering logic, maintaining UI responsiveness during data sorting operations.

Success Metrics (KPIs):

The impact of the dashboard will be quantified by tracking the following key performance indicators:

- Time to Triage:** The objective is to reduce the average time taken from "Alert Received" to "Ticket Created" to **less than 30 seconds**.
- Remediation Rate:** Measuring the percentage of "Critical" findings that are resolved within **24 hours** of their initial appearance on the dashboard.

5. Development Action Items

To transition this prototype into a production-grade enterprise application, the following engineering tasks are required:

1. **RBAC Integration (Security):** The current backend mocks data. It needs to be integrated with the Identity Provider (IdP) to ensure users only see findings for accounts they are authorized to manage (e.g., prod-admin vs. dev-user).
2. **Server-Side Pagination (Scalability):** As findings grow from 100 to 10,000+, client-side filtering will become a bottleneck, then need to move sorting and pagination logic to the backend API.

Demo Application Link : <https://cloud-dashboard-8qmu.vercel.app/>